



DTSC422: Natural Language Processing

FINAL REPORT

Intelligent RAG-Based Knowledge Assistant for Discover India Program Research

December 9th, 2025

Submitted by: Chaitri Nair, Kriti Rai, Mehar Chaudhry, and Samichi Rungta

Submitted to: Professor Manoranjan Dash

Abstract

This project builds an intelligent research-analytics platform for FLAME University's Discover India Program, addressing the difficulty of searching across student research papers. The system integrates an analytics dashboard with a conversational RAG-based chatbot. The dashboard uses NLP to visualize themes, geographic trends, and cultural keywords. The question-answering system uses Mistral embeddings, Pinecone vector search, and cosine similarity to generate accurate, citation-linked responses. Together, these components improve research accessibility for students, faculty, and administrators.

Keywords: RAG, Vector Databases, NLP, Document QA, Cultural Analytics

1. Introduction

1.1 Background & Motivation

One of Flame University’s experiential learning programs is the Discover India Program. The program’s primary focus is on exploring, understanding, and experiencing the rich culture and heritage of India. DIP field reports are long, narrative, and vary in terminology, making keyword search ineffective. Users must skim hundreds of pages to find relevant material, and queries often require synthesizing information across documents. RAG systems offer a solution by combining semantic retrieval with controlled generation, but academic datasets add challenges such as long-document structure, factual accuracy, and the need for verified citations.

1.2 Objectives

1. Build a full pipeline from PDF ingestion to question answering.
2. Enable semantic search via neural embeddings.
3. Ensure answers remain grounded in source documents with citations.
4. Evaluate performance across diverse query types.
5. Provide an analytics dashboard for corpus-level insights.

1.3 Contributions

The project adapts RAG for ethnographic student research, demonstrating that general-purpose embeddings work well for culturally rich Indian content. It integrates citation tracking for academic reliability and uses cost-effective tools (Mistral + Pinecone). We propose evaluation methods for unstructured cultural datasets and build two interfaces: a chatbot for focused queries and a dashboard for exploratory insights.

2. Related Work

2.1 RAG

RAG (Lewis et al., 2020) combines document retrieval with text generation to reduce hallucinations. It retrieves relevant passages and uses them as grounded context for answering questions. While the original work focused on open-domain QA using Wikipedia, we adapted RAG to long academic papers in the Discover India corpus. Our setup uses Mistral for

embeddings and generation and Pinecone for vector search, with an added emphasis on accurate citation, critical in academic use cases.

2.2 Document Question Answering Systems

Early QA systems were extractive, such as SQuAD (Rajpurkar et al., 2016), while systems like DrQA (Chen et al., 2017) added retrieval for open-domain questions. Our system differs by retrieving across 28 documents, generating synthesized answers, and always providing source citations. It also targets a domain-specific cultural dataset rather than broad general knowledge.

2.3 Vector Database

Traditional databases are not designed for high-dimensional similarity search. FAISS (Johnson et al., 2019) improved ANN search but still requires infrastructure management. Managed vector databases such as Pinecone, Weaviate, and Milvus simplify this. We use Pinecone for its serverless design, low maintenance, and easy LangChain integration.

2.4 Analytics

NLP analytics have progressed from basic frequency methods like LDA (Blei et al., 2003) to transformer-based models for contextual understanding. Visualization tools like t-SNE (van der Maaten & Hinton, 2008) support clustering of high-dimensional embeddings. Our dashboard combines keyword categorization, geographic entity extraction, spaCy NER, and t-SNE clustering to provide multiple complementary views of the DIP corpus.

2.5 Educational AI Applications

AI is widely used in tutoring, grading, and academic search (e.g., Semantic Scholar). However, general-purpose LLMs struggle with institution-specific content and citation accuracy. Our system offers a lightweight way to make internal research accessible, with grounded responses and verifiable citations. Unlike fine-tuned models, specialization is achieved through retrieval rather than training, enabling fast and low-cost deployment for experiential learning datasets.

3. Methodology

The methodology covers the overall system architecture, data pipeline, vector database configuration, embedding generation, and the Retrieval-Augmented Generation (RAG) process for final response generation.

3.1 System Architecture

The system follows a Retrieval-Augmented Generation (RAG) pipeline designed for document ingestion, semantic indexing, and high-quality answer generation (*Refer to Appendix for Architecture diagram*). The overall architecture consists of four stages: document processing, vectorization, semantic search, and response construction. The pipeline begins with input of documents that are loaded from a directory using LangChain's DirectoryLoader, which supports multiple file formats including PDFs, text files, and Word documents. The system extracts text content while preserving document metadata such as source file names and structural information.

Once loaded, documents undergo a chunking process essential for efficient retrieval and processing. The RecursiveCharacterTextSplitter breaks down the research papers into manageable segments of 1000 characters with a 20-character overlap between chunks. This overlap ensures that contextual information at chunk boundaries is preserved, preventing the loss of meaning across splits. The algorithm tries to split at natural boundaries in order: double newline (paragraphs), single newline (sentences), space (words), then characters as last resort. This strategy balances two needs: chunks must be small enough for efficient embedding and retrieval, and large enough to maintain semantic coherence.

Each text chunk is transformed into a high-dimensional vector representation using Mistral AI's embedding model (mistral-embed). These embeddings capture the semantic meaning of text in a 1024-dimensional vector space, where similar concepts are positioned closer together. This mathematical representation allows for semantic search capabilities that go beyond simple keyword matching.

The generated embeddings are stored in Pinecone, a vector database optimized for similarity search at scale. Pinecone uses a serverless architecture hosted on AWS, giving low-latency access and automatic scaling. The database indexes vectors using cosine similarity as the distance metric, which allows rapid retrieval of semantically related document chunks. When a user asks a question, it undergoes the same embedding as the document chunks. The system then performs a similarity search in Pinecone to identify the most relevant chunks. These retrieved chunks serve as context for Mistral AI's language model (mistral-medium), which outputs a contextually-relevant response and cites specific source documents.

The end-to-end pipeline follows this sequence:

User Query → Embedding → Similarity Search → Context Retrieval → Prompt Assembly → LLM Generation → Response with Citations.

This architecture ensures that all responses are relevant in the actual content of the research papers, preventing the hallucination.

3.2 Dataset

Data Collection:

We collected papers through two channels: directly from peers who participated in DIP, and from FLAME University's library repository. This ensured diverse coverage across different years and regions.

Dataset Statistics:

Total Papers: 22

Format: PDF documents

Length: 100-200 pages per paper

Time Range: 2018-2023

Content: Narrative text, photographs, maps, tables

Thematic Coverage:

Papers cover nine cultural domains: Architecture & Monuments, Performing Arts, Visual Arts & Crafts, Religious Traditions, Festivals & Celebrations, Food & Cuisine, Traditional Livelihoods, Social Customs, and Language & Literature.

Geographic Distribution:

Research covered 15-20 Indian states with concentration in North (Rajasthan, UP, Uttarakhand), South (Tamil Nadu, Kerala, Karnataka), and West India (Maharashtra, Gujarat, Goa). Northeast states and tribal areas were underrepresented.

3.3 Vector Database

The system utilizes Pinecone as its vector database solution, selected for its specialized design for similarity search operations and its ability to handle high-dimensional vector data efficiently.

3.3.1 Pinecone Configuration

A dedicated index named "langchain-mistral-demo" is created to store the embedded document chunks. The index is configured with a dimensionality of 1024 to match the output dimensions of the Mistral embedding model. This dimensionality represents a balance between expressive power (higher dimensions can capture more nuanced semantic relationships) and computational efficiency.

Pinecone preserves associated metadata for each chunk, including the source document filename and the original text content. This metadata is crucial for the citation system, allowing the response generation module to attribute information to specific source papers.

3.3.2 Cosine Similarity

Cosine similarity measures the cosine of the angle between two vectors in the embedding space, yielding values between -1 (opposite) and 1 (identical). Unlike Euclidean distance, which considers the magnitude of vectors, cosine similarity focuses purely on directional alignment. This property is particularly valuable for text embeddings, where the direction in vector space encodes semantic meaning while magnitude may reflect factors like document length rather than semantic content.

Cosine similarity's magnitude independence ensures that a short phrase and a longer passage expressing the same idea can still be recognized as semantically similar. This characteristic is crucial when comparing user queries (typically brief) against document chunks (more extensive).

Extensive research in information retrieval and natural language processing has demonstrated that cosine similarity consistently outperforms alternative metrics for text-based semantic search tasks, particularly when working with embeddings from modern language models.

3.4 Retrieval Augmented Generation

This stage works by retrieving the most relevant chunks from Pinecone and feeding them into a LangChain question-answering chain built on top of the Mistral LLM. The LLM does not operate on the full corpus. Instead, it only sees a compact context of the top-ranked retrieved segments. This design forces the model to stay anchored in the actual documents and prevents hallucinated facts.

The response generation module is flexible, the retrieval, context assembly, prompting, and final answer construction are independent components that can be improved without altering the architecture. This makes the overall system maintainable and extensible, especially when evaluating or upgrading embedding models or vector database configurations.

3.4.1 Context Assembly

When a user query is processed, the system retrieves the top-k most similar document chunks from Pinecone (k=4). This value is calibrated to provide sufficient context without overwhelming the language model's context window or introducing excessive noise. The retrieved chunks are ranked by cosine similarity score, ensuring the most relevant information is prioritized.

Retrieved chunks may occasionally come from the same source document or contain overlapping information. The context assembly process removes redundant information while maintaining the most relevant instances. Chunks are organized by relevance score, placing the most relevant information earlier in the context, as language models tend to assign greater weight to earlier context.

Throughout assembly, the system maintains careful tracking of which chunks originated from which source documents. This metadata is preserved for the citation system. The assembled

context must fit within the language model's maximum context window (typically 4,000-8,000 tokens for the models used). The assembly process monitors token counts, ensuring that the query, context, system instructions, and response buffer all fit comfortably within limits.

3.4.2 Prompt Engineering

Prompt engineering transforms the raw retrieved context and user query into an optimally structured input for the language model. Retrieved document chunks are presented in a structured format, clearly delineated from the user query. Each chunk is accompanied by its source document identifier, enabling the model to attribute specific claims to specific sources during generation. The user's question is restated clearly, often with slight rephrasing to optimize for the model's comprehension. The prompt structure signals to the model that it should synthesize information from the preceding context to answer this specific query.

The prompt includes explicit instructions about desired output format, including guidelines for citation formatting, response structure, and tone. For the DIP analytics platform, the model is instructed to adopt an informative, academic tone appropriate for research discussions. This can significantly improve consistency in the model's output format.

3.4.3 LLM Generation

The generation phase leverages Mistral AI's language model (mistral-medium) to generate responses. The mistral-medium model was chosen for its balance between response quality, speed, and cost. It provides strong performance on question-answering tasks while maintaining reasonable inference latency (typically 2-5 seconds for responses of 200-300 tokens).

The model employs a relatively low temperature setting (typically 0.3-0.5) to prioritize factual accuracy and consistency over creative variation. Lower temperatures reduce the probability of the model generating speculative or unsupported claims, reinforcing the system's commitment to grounded responses.

The model generates responses token-by-token, with each token's probability conditioned on the prompt context and all previously generated tokens. This autoregressive generation continues until the model emits a stop token or reaches a predetermined maximum length.

The underlying transformer architecture allows the model to attend to all parts of the input context simultaneously, weighing different chunks' relevance for each generated token. This enables the model to synthesize information from multiple sources when constructing comprehensive answers.

3.4.4 Citation System

The citation system ensures transparency and traceability, unlike usual AI assistants. As the language model generates its response, it internally forms associations between generated content and the source chunks that informed that content. After generation, the system performs post-processing to extract and format citations. It identifies which of the retrieved document chunks were most relevant to the generated response by analyzing attention scores or, in simpler implementations, by listing all source documents of the retrieved chunks.

The citation system enables users to verify claims by consulting the original source documents. This verification capability is crucial for academic and research applications, where users need to assess the reliability and context of information independently.

3.5 Analytics Dashboard

(Refer to Appendix for Architecture Diagram)

3.5.1 Thematic Analysis

We categorized the papers into nine predefined cultural themes using keyword-based categorization where each category has curated keywords. We chose this over ML methods like LDA, and neural classification because it is more transparent, explainable, and aligned with known DIP categories. This method was also more computationally efficient and is easy to maintain. We also added bar charts for frequency distribution, pie charts for proportional theme coverage, and document level tables showing each paper's primary topic.

3.5.2 Geographic Analysis

We wanted to identify which Indian states/UTs appear in research as well as visualize the geographic coverage. This would make it easier for future groups to see what areas have already been researched, and what areas have more scope. We used pattern matching for all 36 Indian

states and UTs. We used pattern matching over other methods like NER because NER models often struggle with multi-word names such as Himachal Pradesh. Our pattern matching model also accounts for abbreviations such as UP for Uttar Pradesh. This then cross tabulates the states with the topics to reveal patterns and insights into the data. For the visualizations, we used bar charts for top states, pie charts for distribution, and state-topic heatmaps showing thematic associations per region.

3.5.3 Cultural Insights

We wanted to extract culturally significant keywords and named entities using keyword extractions, word clouds, and NER. We use term frequency (not TF-IDF) for keyword extraction because we want corpus-level patterns, terms frequent across all papers indicate shared research focus. Minimum word length of 4 characters filters function words while retaining content words. We created word clouds of 80 words to balance detail with readability. The word clouds allow the user to see the most common culturally significant words. We extract culturally relevant entity types: GPE (cities, states), LOC (rivers, mountains), PERSON (artists, leaders), ORG (temples, institutions), EVENT (festivals), and WORK_OF_ART (dance forms, art styles). This lets the user understand the most common cultural topics. In terms of visualizations, we used pie charts for entity type distribution, bar charts for top locations/persons/events, and filterable tables for all entities.

3.5.4 Document Clustering

Finally, the dashboard includes document clustering, which was done to discover latent thematic groupings through unsupervised learning and to visualize document relationships. We used the same embeddings as the chatbot, the mistral embedding model. We then used t-SNE to reduce the embedding to a 2D vector for visualization. We then used K-means to cluster the data and then visualize the clusters as scatter plots.

4. Experiments and Analysis

4.1 Evaluation Methodology

For the user-facing evaluation we created 30 authentic, domain-relevant questions reflecting the types of queries faculty and students actually ask (examples below). The questions were grouped

into categories to analyze strengths and weaknesses across common use cases, and two independent reviewers judged answer quality based on a three-step rubric:

1. **Relevance:** Did the answer respond to the question asked?
2. **Grounding:** Did the answer cite and rely on retrieved document chunks?
3. **Accuracy / Usefulness:** Was the answer factually correct given the cited sources and useful for the user's goal?

Because DIP research contains narrative, field notes and local terms, we prioritized real-world applicability over purely quantitative metrics.

4.2 Analytics Dashboard

Using our analytics dashboard, we examined overall patterns in the DIP papers across themes, geography, keywords, sentiment, and document similarity. Most papers focus on Visual Arts & Crafts, Food & Cuisine, and Architecture, while areas like Performing Arts, Festivals, and Social Customs are underrepresented. A few states, Goa, Delhi, Kerala, Maharashtra, and Rajasthan dominate the geographic mentions, showing that projects cluster in more accessible locations. The topic-state heatmap highlights strong associations, such as crafts and food in Goa and architecture in Rajasthan.

Cultural keyword counts and word clouds reflect frequent references to heritage, crafts, dance, and music, along with common descriptive language. Entity co-occurrence graphs reveal central nodes like Tamil Nadu, Delhi, UNESCO, Hampi, and Shiva, showing how cultural concepts connect across papers.

Document similarity analysis using embeddings and t-SNE produced clear clusters, mostly around architectural topics, confirming that chunking and embeddings worked well. Sentiment remained largely neutral. Overall, the dashboard provided a clear view of covered and underexplored areas and validated the system's analytical accuracy.

(Refer to Appendix for all graphs/figures)

4.3 Chatbot Performance

Evaluation of chatbot performance focused on the relevance, grounding, and clarity of generated responses. The corpus used represents a diverse collection of research themes across different regions of India, providing a solid test environment for retrieval-based question answering.

4.3.1 Query Categories

To assess the system across different content dimensions, queries were grouped into several categories:

The evaluation queries were grouped as follows:

- **Location-specific queries** (e.g., “Which projects were conducted in Kerala?”; “What does the Indore report say about X?”).
- **Cultural/thematic queries** (e.g., “Explain Goan ‘mando’”, “Describe traditional Kundan jewellery practices”).
- **Architectural/heritage queries** (e.g., “Which temples or forts are documented for Rajasthan?”).
- **Methodology/questions about research practice** (e.g., “How were interviews conducted in the Murthal dhaba study?”).
- **Cross-document synthesis** (multi-document summarization such as “Compare festival practices across Goa and Tamil Nadu”).

These categories mirror the primary use cases: students preparing fieldwork, faculty planning new projects, and archivists exploring research coverage.

4.3.2 Results

Across all categories, the chatbot produced responses that were generally accurate, well grounded, and backed by relevant source citations.

For location-specific questions, the system correctly retrieved documents linked to the corresponding city or region. For instance, a query about "the project done in Indore" resulted in context pulled from the actual Indore project report. This shows how effective the embeddings and vector indexing are.

Responses related to cultural content, such as festivals or art forms, showed strong summarization skills while staying true to the retrieved context. The clear instruction to avoid

guessing significantly reduced mistakes. When the context did not provide enough detail, the model responsibly answered with variations of “I don’t know.”

The addition of source citations boosts trust and clarity, which is important for academic use cases where proper attribution matters.

5. Discussion

5.1 Chatbot performance evaluation

This project shows that a Retrieval-Augmented Generation approach can greatly improve access to academic research archives. The combination of Mistral embeddings, Pinecone vector storage, and prompt-guided LLM reasoning created a reliable and academically suitable question-answering system. The analytics dashboard is also valuable. It provides insights based on themes, geography, culture, and clustering. This two-system design supports both detailed question answering and a broader understanding of research.

An important finding is that traditional rule-based methods, such as keyword-based topic classification and curated cultural lexicons, still work well with LLM-based techniques. They offer clarity and control.

5.2 Limitations

Despite its strengths, the system has several limitations. Rule-based thematic classification may overlook nuanced or implicit topics that do not depend on explicit keywords. Sentiment analysis relies on a simple lexicon and fails to capture deeper tonal subtleties. Entity extraction uses a general-purpose spaCy model, which may not completely recognize India-specific cultural terms. Scalability issues arise in the clustering and t-SNE visualization components. The system also depends on external APIs, and their latency or availability may occasionally impact performance.

5.3 Learnings

The implementation process showed how important prompt engineering is for managing LLM behavior and reducing hallucinations. Session management and clear user feedback in Streamlit greatly improved the user experience.

The project also illustrated the benefits of merging modern NLP techniques with traditional analytics methods. The addition of interpretability tools, such as citation display and visual dashboards, was crucial for building user trust. This work lays a solid foundation for managing institutional knowledge.

6. Conclusion & Future Work

6.1 Conclusion

This project successfully developed an AI-driven system for interacting with and analyzing Discover India Program research reports. The chatbot allows for accurate, citation-backed question answering. The analytics dashboard offers insights into themes, locations, cultural elements, and textual patterns within the corpus.

The system not only makes previous research easier to access but also serves as a teaching tool for students starting new DIP projects. By combining modern LLM technologies with traditional NLP and visualization methods, the project creates a solid foundation for managing institutional knowledge.

6.2 Future Work

Several directions exist for improving the system:

- Short-term: Improve sentiment analysis with transformer-based models and include richer metadata in document representations.
- Medium-term: Add multilingual support, topic modeling (e.g., BERTopic), and domain-adapted NER models.
- Long-term: Develop the platform into a research knowledge portal for the entire university, with better search, recommendation systems, and long-term analytics.

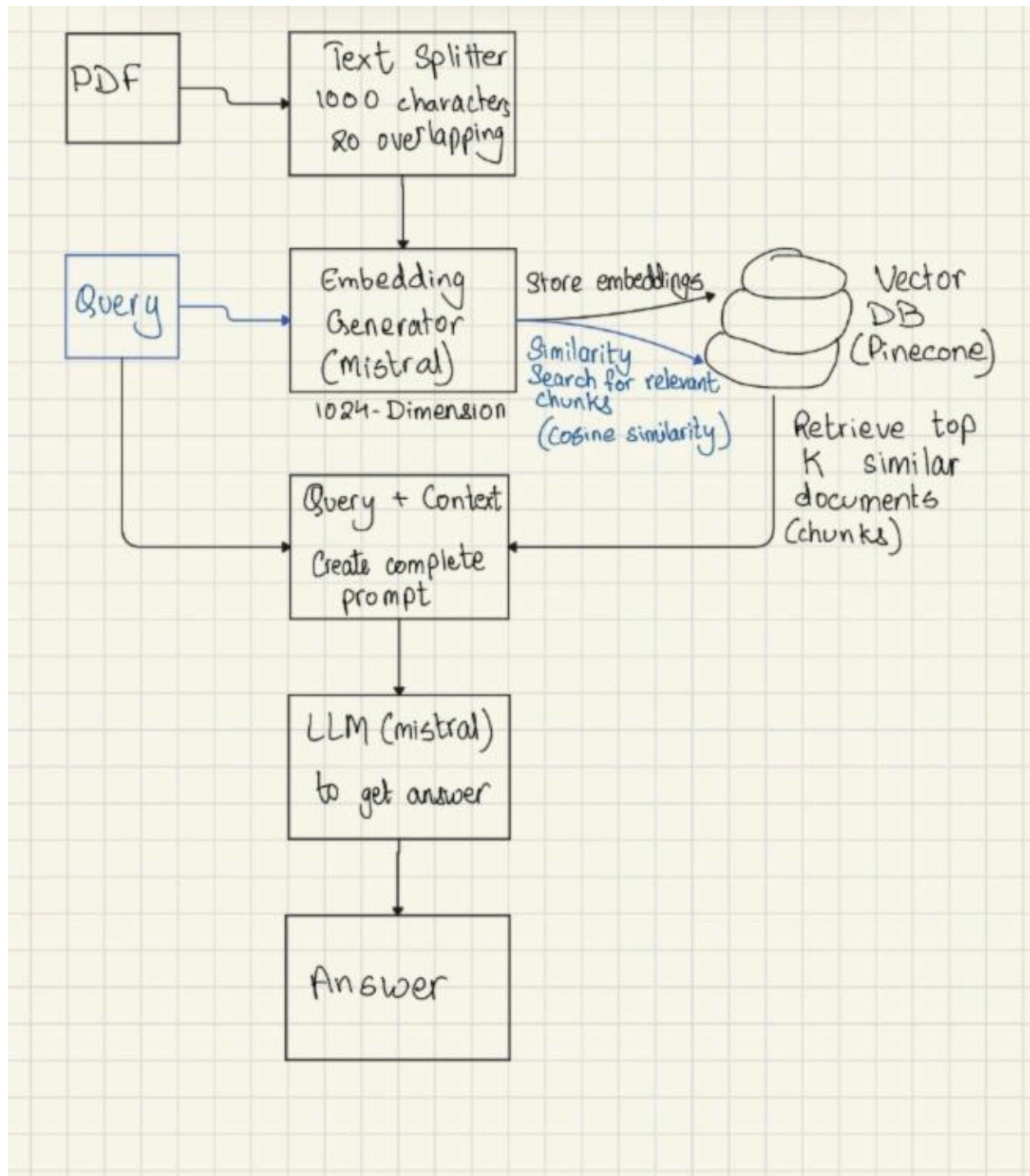
6.3 Broader Impact

By turning static DIP reports into an interactive, searchable, and interpretable knowledge resource, this project strengthens the academic community at FLAME University. It supports research continuity, improves discoverability, and encourages student engagement with India's cultural and historical richness. More broadly, the system acts as a model for how educational institutions can preserve and use student-generated research on a large scale.

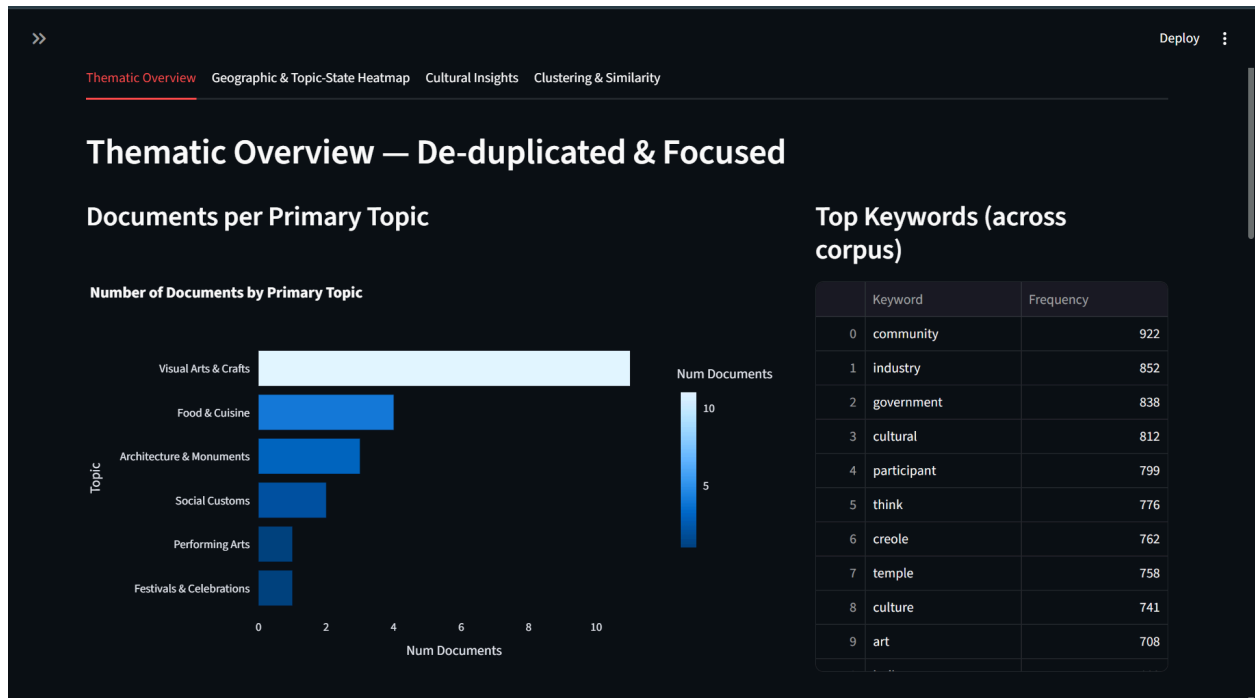
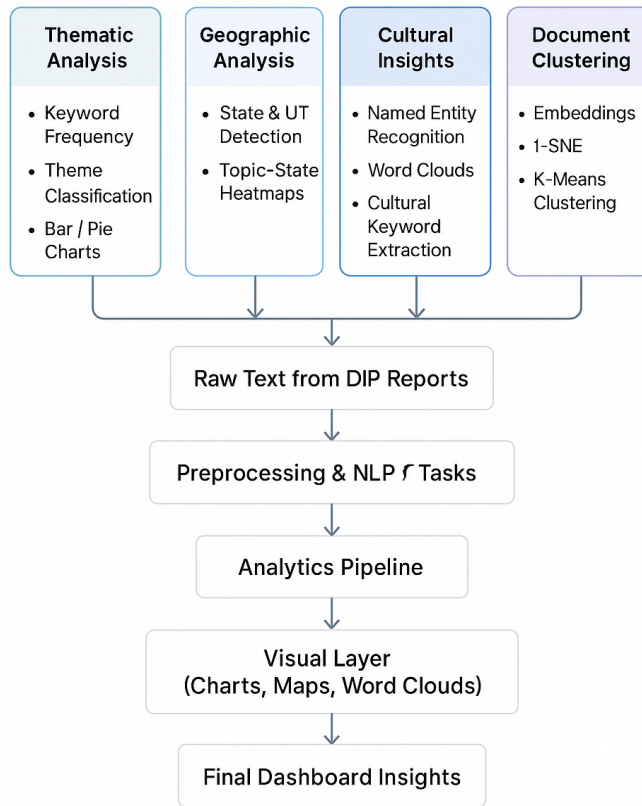
7. Bibliography

- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3, 993–1022. <https://dl.acm.org/doi/10.5555/944919.944937>
- Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). Reading Wikipedia to answer open-domain questions. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1870–1879). Association for Computational Linguistics. <https://aclanthology.org/P17-1171/>
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://ieeexplore.ieee.org/document/8733051>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 9459–9474). <https://doi.org/10.48550/arXiv.2005.11401>
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 2383–2392). Association for Computational Linguistics. <https://doi.org/10.48550/arXiv.1606.05250>
- Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605. <https://jmlr.org/papers/v9/vandermaaten08a.html>

8. Appendix



Analytics Dashboard Architecture

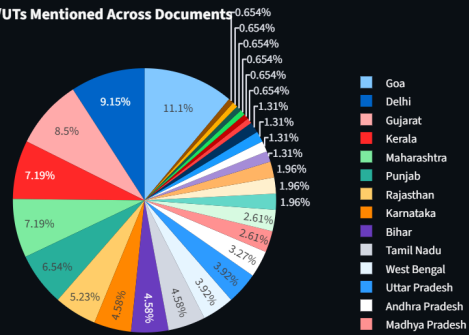


Geographic Distribution & Topic-State Heatmap

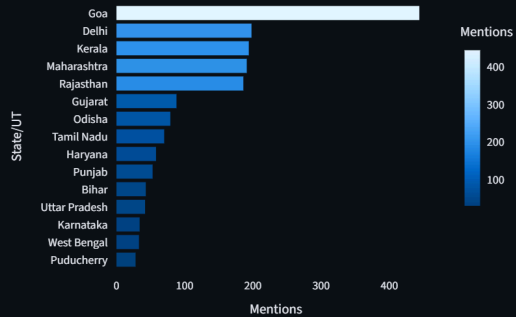
States/UTs: In how many papers (pie)

States/UTs: Mention frequency (bar)

States/UTs Mentioned Across Documents

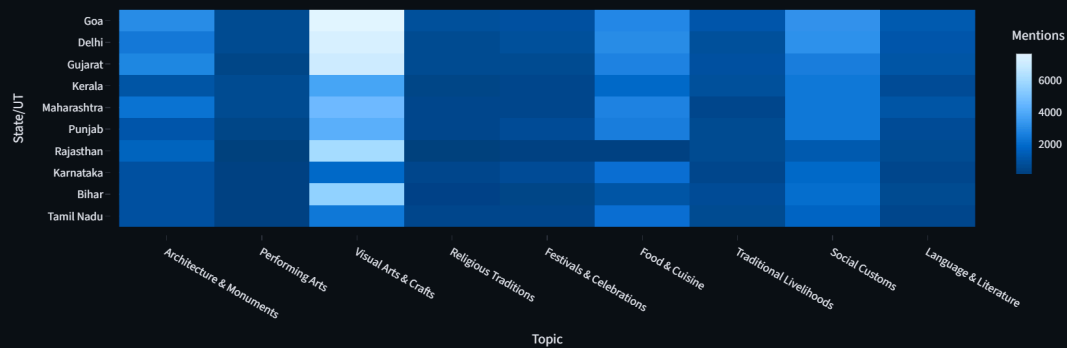


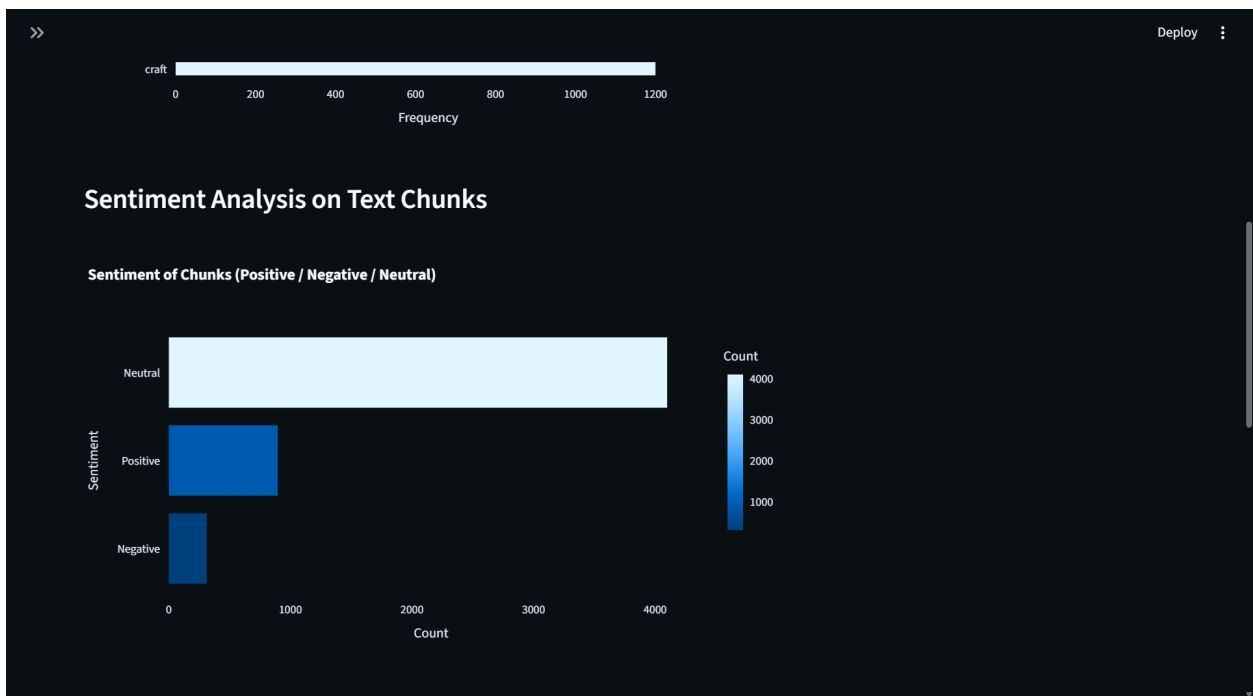
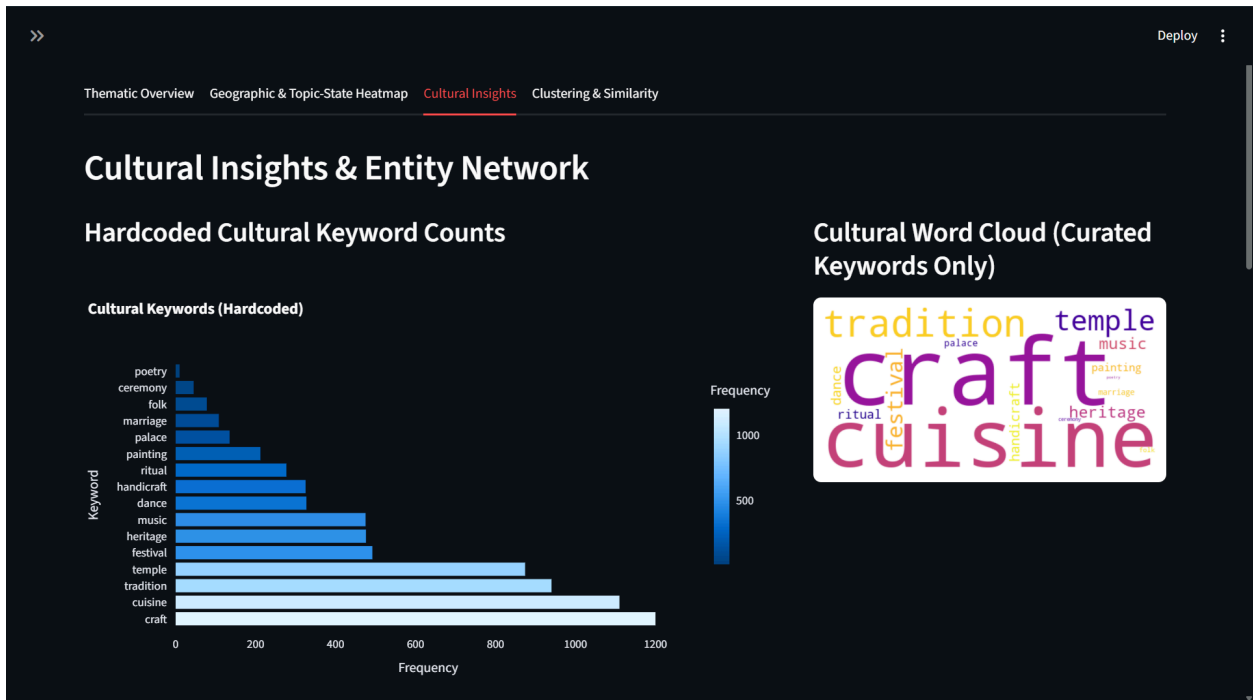
Top States by Mentions

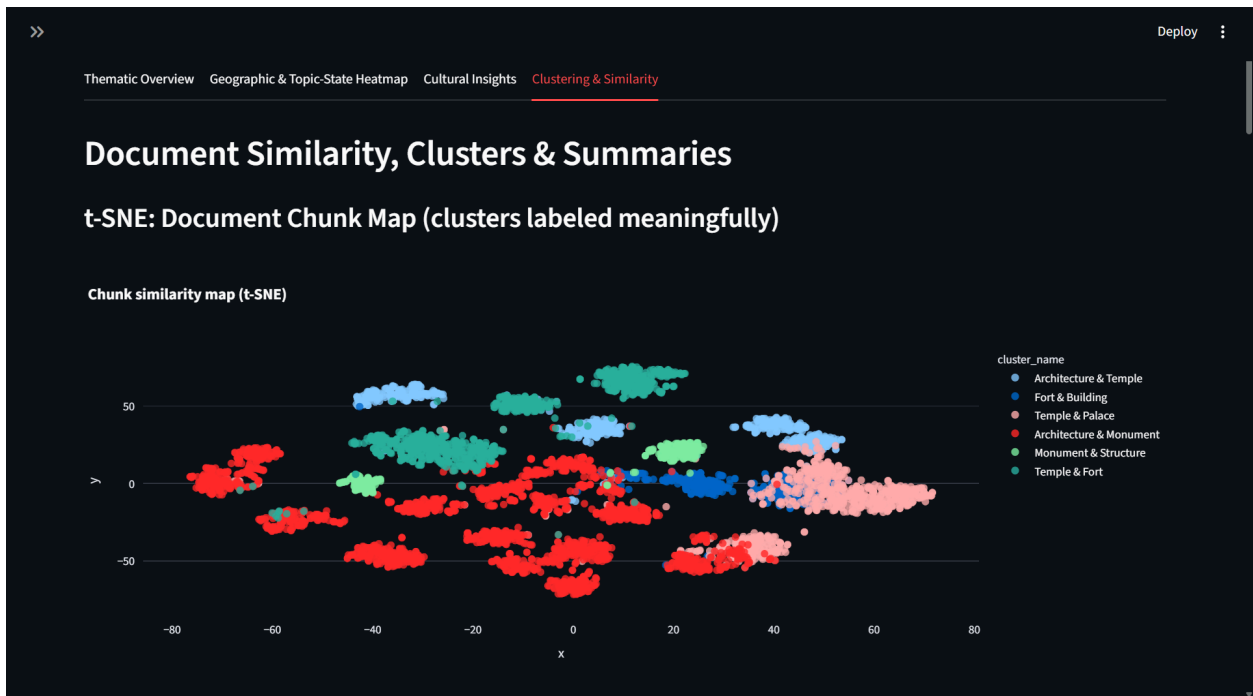
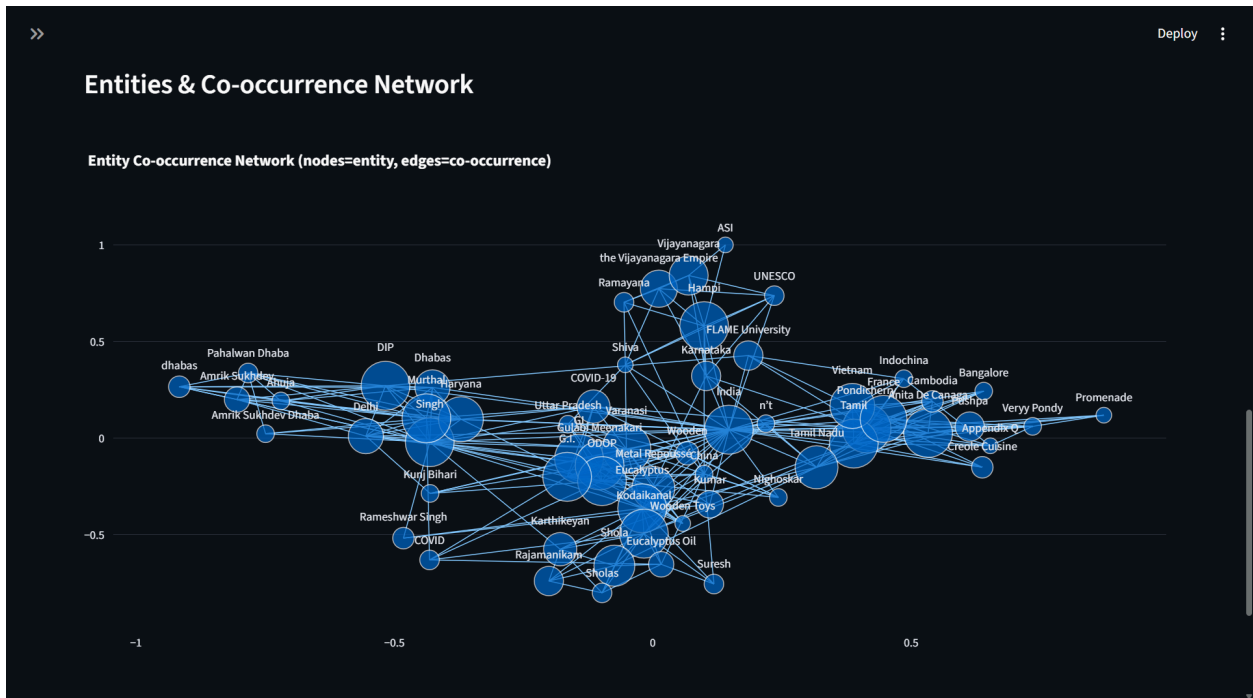


Topic-State Heatmap (Top 10 states)

Topic mentions per State (Top 10 states)





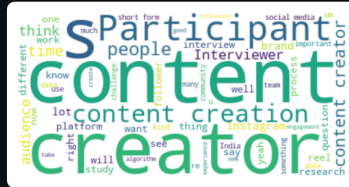


Cluster Word Clouds

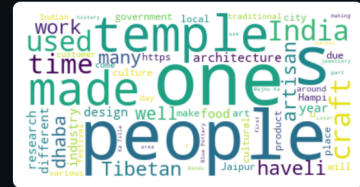
Cluster 0 – Monument & Structure



Cluster 1 — Fort & Building



Cluster 2 — Architecture & Monument



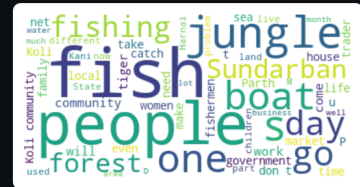
Cluster 3 — Temple & Palace



Cluster 4 — Architecture & Temple



Cluster 5 – Temple & Fort



Explore a Cluster